Pattern Recognition

Performance comparison between OpenMP and CUDA

Albi Sadikaj
Cosimo Sgambelluri

April 2026

Pattern Recognition in Time Series

Pattern Recognition

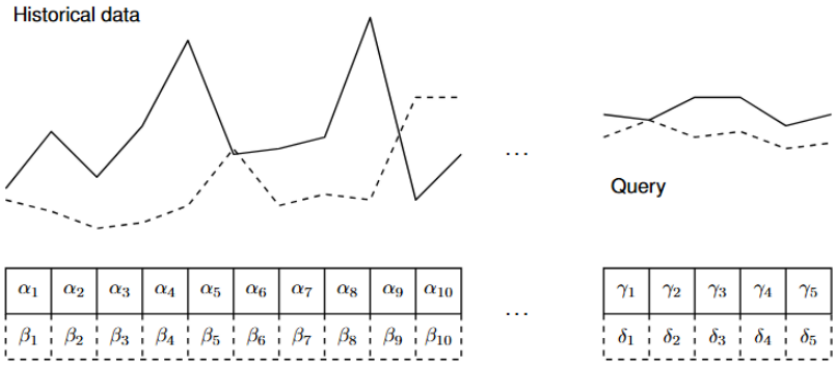
The primary goal of pattern recognition in time series analysis is to locate a specific query sequence within a much larger dataset.

The method

Because perfect data alignment is statistically improbable, the process relies on proximity metrics, specifically the Sum of Absolute Differences (SAD), to quantify similarity and isolate the closest match.

Why it is computationally expensive

- **Massive operations volume**: a sliding window approach has a time complexity of $O(N \times M \times Q \times C)$. For example, evaluating four queries of length 819 against a dataset of 2M points requires nearly 33B floating-point operations.
- **Memory bandwidth bottlenecks:** because sliding windows continuously evaluate overlapping data points, a standard implementation repeatedly fetches the same values.



SAD: Sum of Absolute Differences

The similarity metric used in this project is SAD or Sum of Absolute Differences. It consists in calculating the absolute differences between overlapping data points and summing them together.

- It compares a sliding window of the time series against the query point-by-point.
- A lower SAD value indicates a higher degree of similarity. A score of exactly zero represents an identical match.

$$SAD(i) = \sum_{j=0}^{M-1} \left| T_{i+j} - Q_j \right|$$

Implementations

Given the nature of the problem, it is possible to parallelize in different ways. Two primary parallelized alternatives were implemented and analyzed:

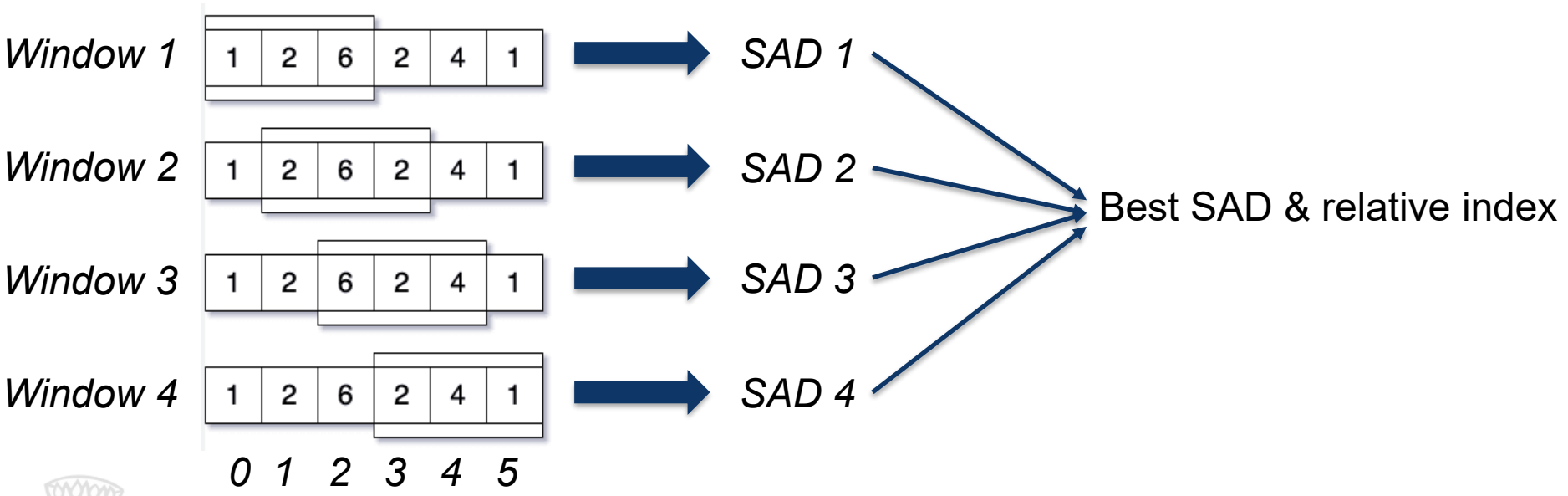
Multi-core CPU Approach: introduces thread-level parallelism by distributing the sliding window search space across available CPU cores.

CUDA GPU Approach: maps the problem space onto a 2-dimensional execution grid, allowing multiple queries to be evaluated simultaneously.

All our implementations execute multiple independent queries across a multichannel synthetic dataset.

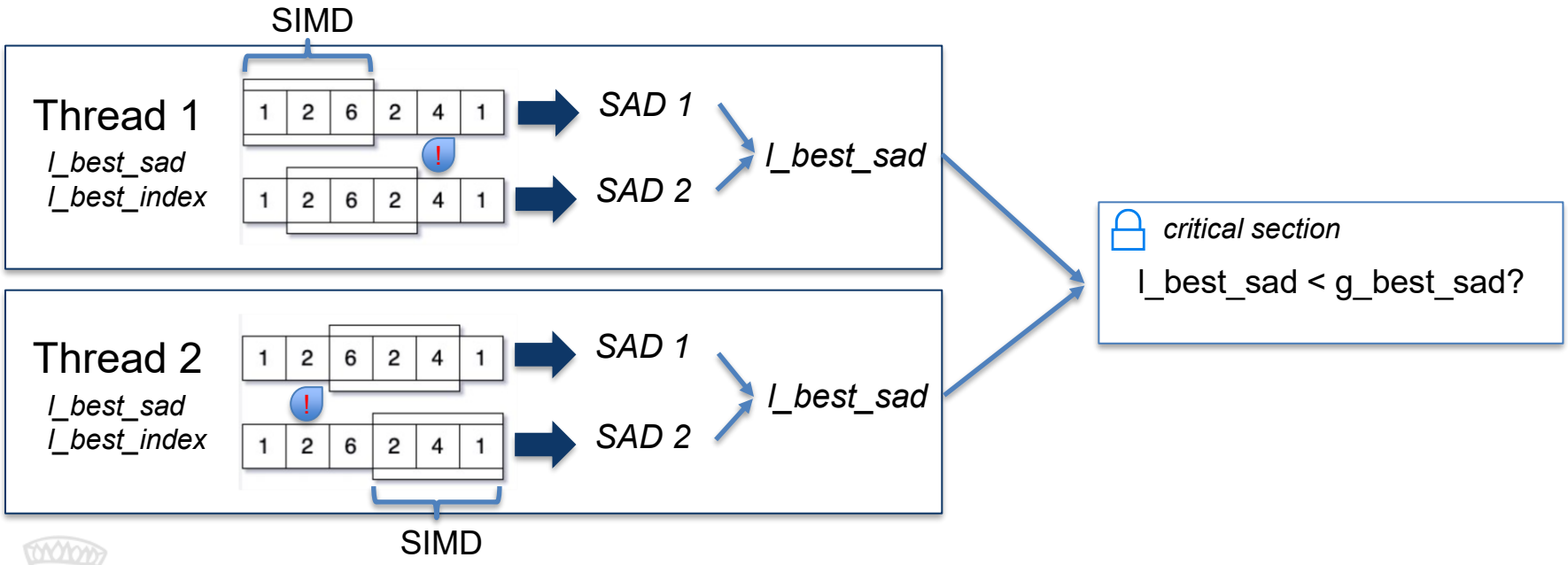
Sequential Implementation

The sequential algorithm relies on a standard sliding window approach, sequentially iterating through the dataset to compute the complete SAD for every single window. Each window has the size of the current query.

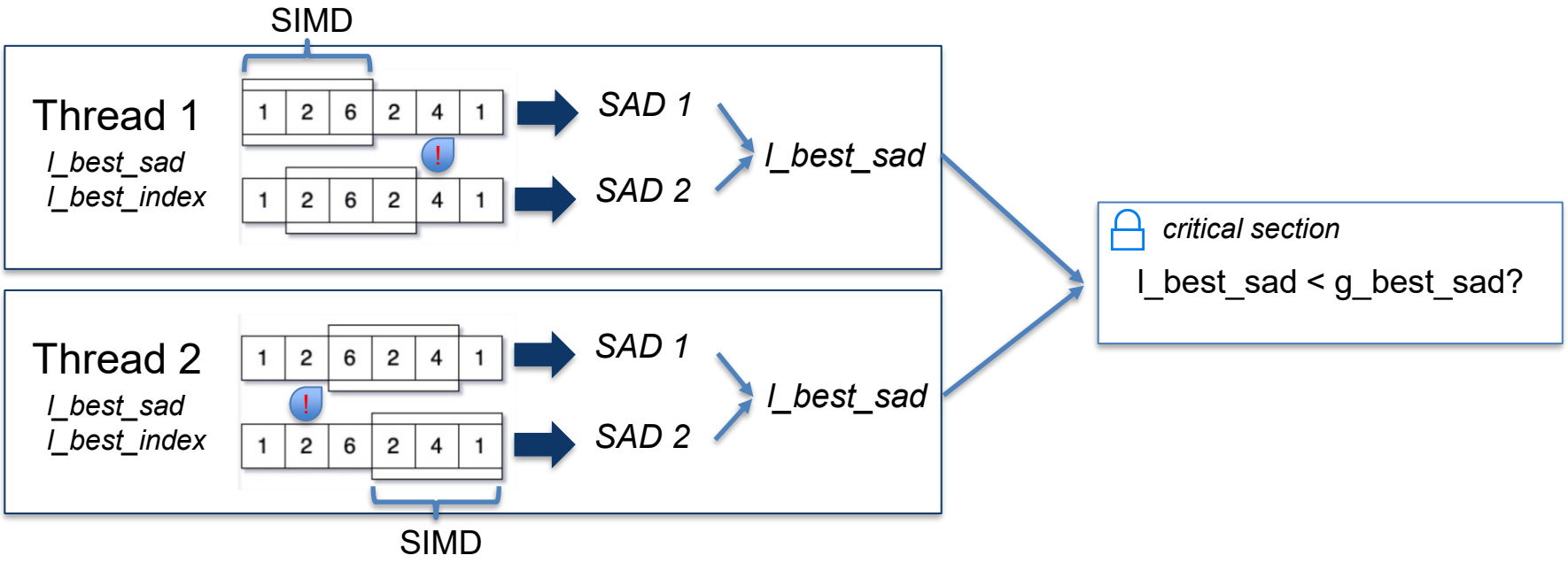


Parallel Version (OMP)

Parallelization is applied across the search space of a single dataset channel. Queries are distributed across threads using dynamic window scheduling, with each thread privately tracking local best matches before safely merging them via a critical section.



Parallel Version (OMP)



To save compute cycles, an ***early abandonment*** check at SIMD boundaries immediately halts the calculation of any window whose partial SAD exceeds the thread's current best.

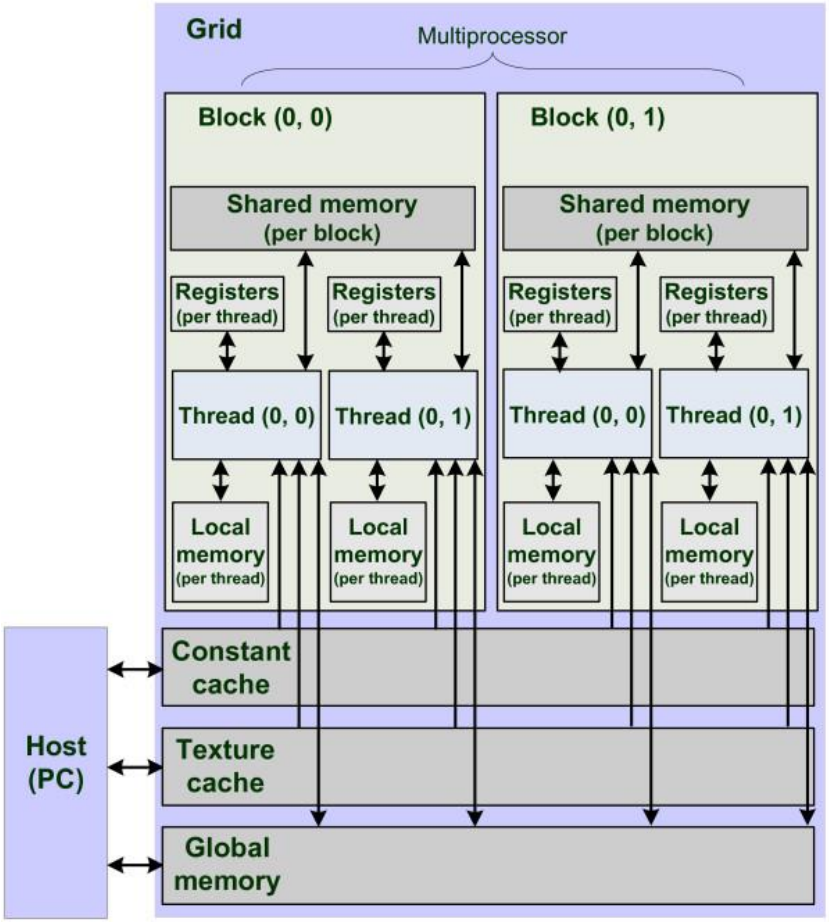
Parallel Version (CUDA)

CUDA threads have access to different kinds of memory:

- Private local memory
- Block shared memory
- Global memory

Each **Streaming Multiprocessor** (SM) has also access to a constant memory, which the kernel can only read.

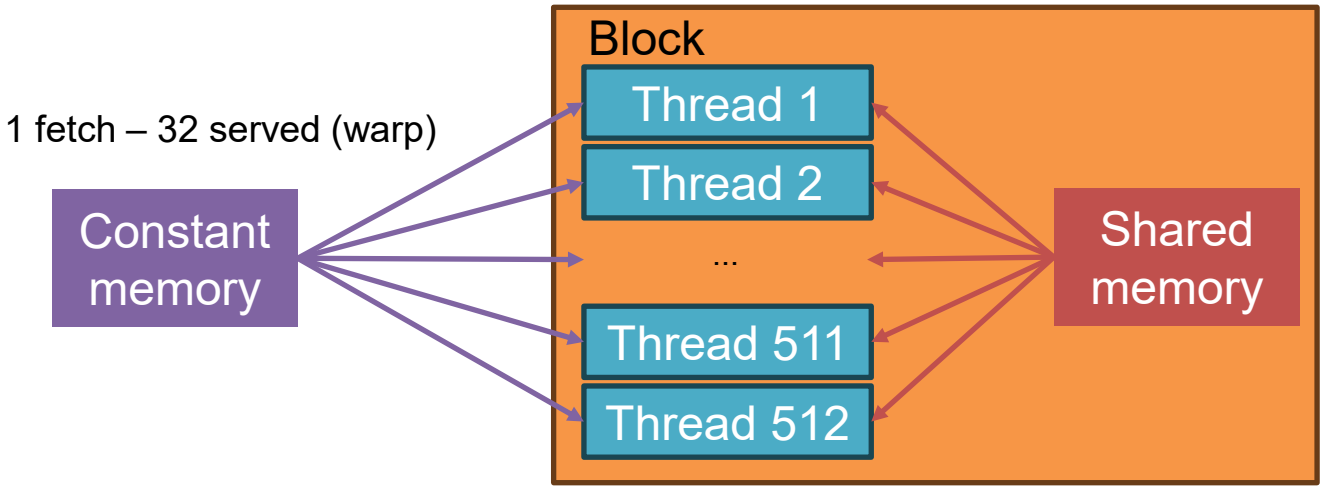
Since our GPU's SMs have a limit of 1536 threads each, we opted to use a block size of 512, allowing each SM to have exactly 3 blocks.



Parallel Version (CUDA)

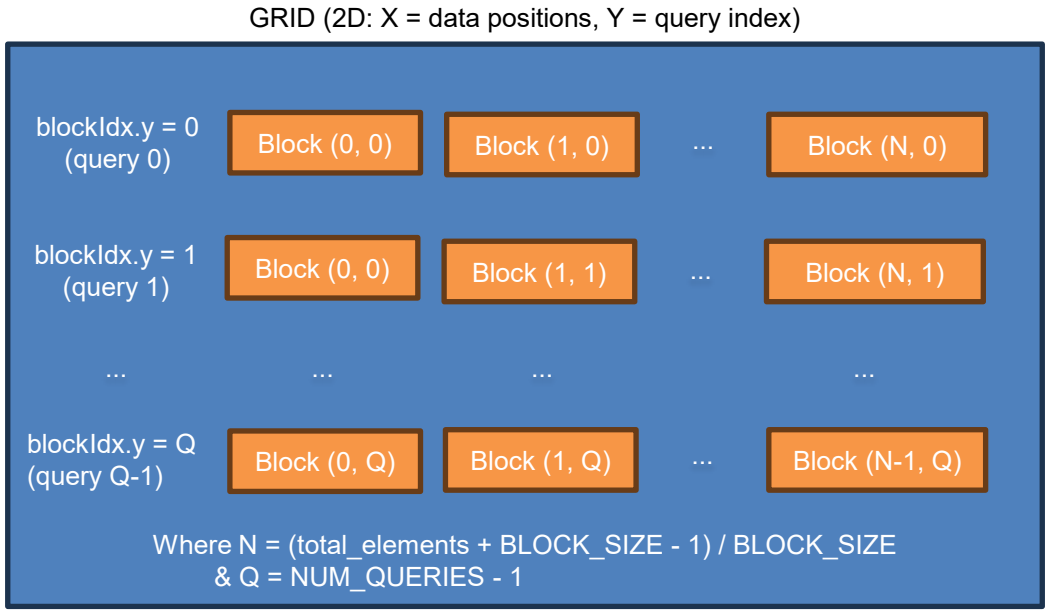
In our attempt to squeeze as much performance as possible we opted for the following memory strategy:

- **Queries → constant memory**: enables hardware broadcast (1 fetch serves 32 threads in a warp instantly).
- **Historical data → shared memory**: ensures highly efficient, conflict-free parallel memory access.

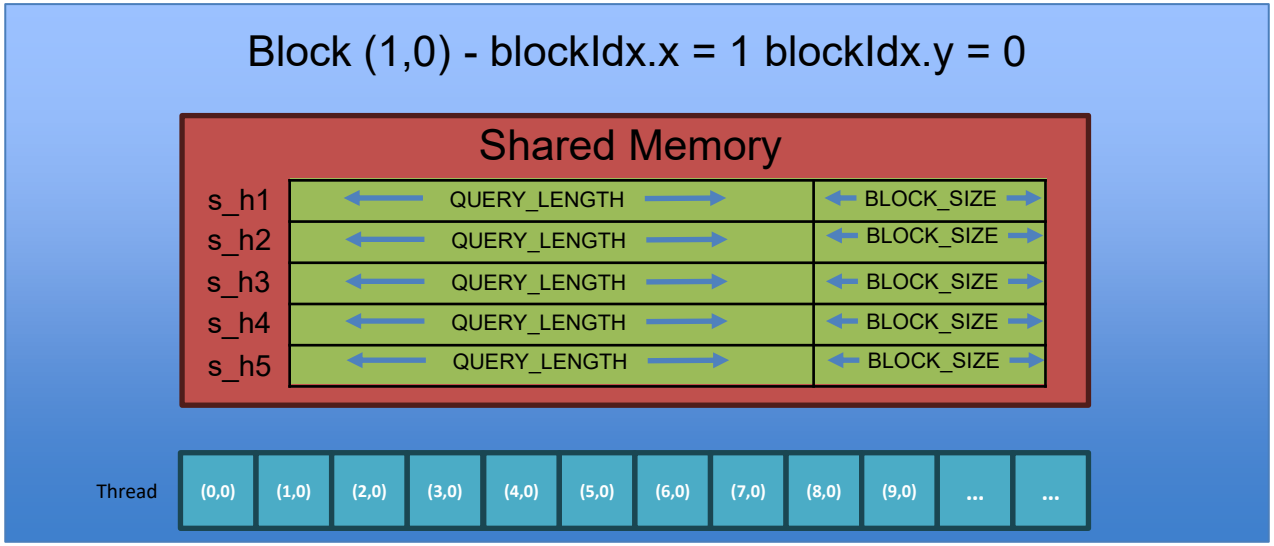


Parallel Version (CUDA)

1. **Initialization:** threads within a block work together to load the required time series data into shared memory, waiting at a synchronization barrier until the entire block is ready.
2. **Parallel computation:** each thread independently calculates the full SAD for its assigned dataset window and query, writing the final score directly to global memory.
3. **CUDA advantage:** by mapping the search space to a massive 2D grid, **all** concurrent queries and windows are evaluated at the exact same time.



Parallel Version (CUDA)



Each thread T_i:

- Reads s_h#[i … i + QUERY_LENGTH-1]
- Reads c_q#[tid_y * QUERY_LENGTH … + QUERY_LENGTH-1]
- Writes 5 SADs in d_results_c#[tid_y * #windows + tid_x]

Benchmark Setup

Parameters Tested

Thread Count: 1 to 16 threads (OMP only)
Query Length: 128, 256, 512, 819
Query Count: 1, 2, 3, 4

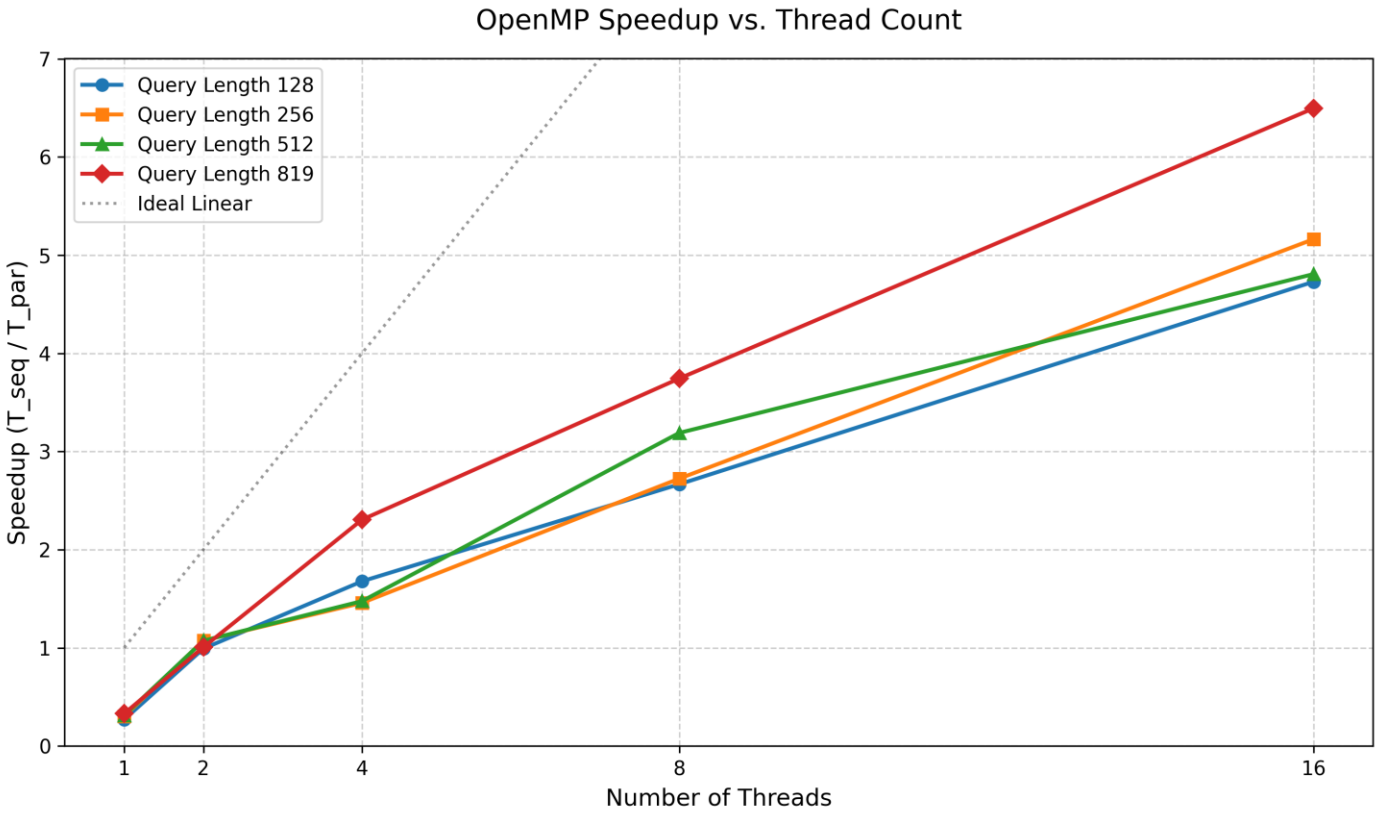
Hardware Setup

CPU: AMD Ryzen 7 7840HS (8 cores)
GPU: NVIDIA RTX 4060 Mobile 8 GB
Method:

- OpenMP: 10 averaged runs + 2 warmups
- CUDA: 100 averaged runs + 10 warmups

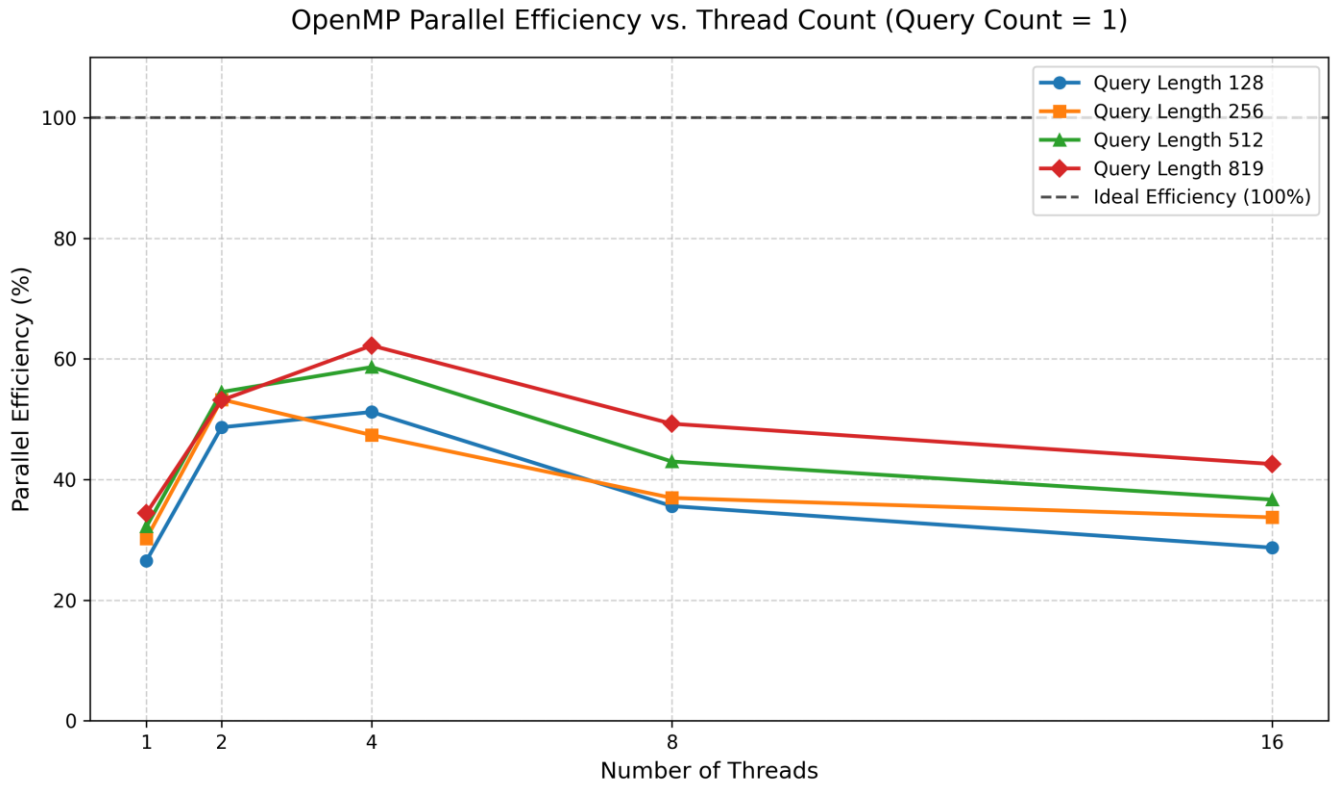
Results: OMP Speedup

The following graph shows the speedup achieved by the OMP parallel version. As we can see, when T=1, OMP is circa 3× slower than the sequential baseline. As thread count increases, so does the speedup until the peak speedup of 6.81× with 16 threads.



Results: Efficiency

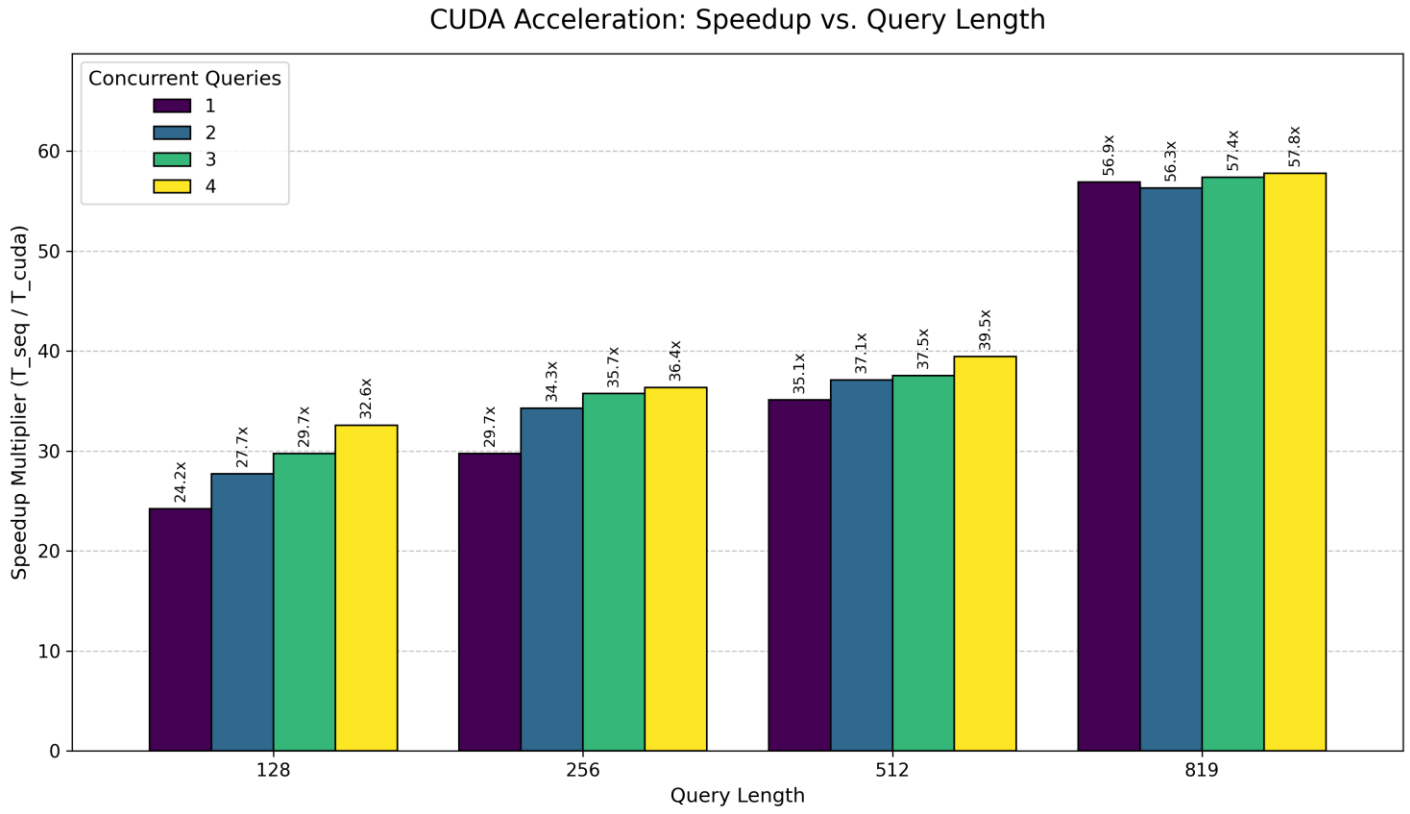
Longer queries (819 elements) consistently perform best. A higher ratio of useful math to parallel overhead keeps the CPU cores better utilized. On the other hand, fork/joining penalties and early abandonment uneven load balancing causes efficiency to degrade as threads number go up.



Results: CUDA Speedup

The GPU implementation outperforms the sequential baseline across all configurations, reaching nearly a 58× speedup.

- **Scaling with query length:** speedup increases as queries get longer.
- **Scaling with concurrent queries:** unlike OpenMP, CUDA has a near-constant speedup multiplier as the number of concurrent queries increases.

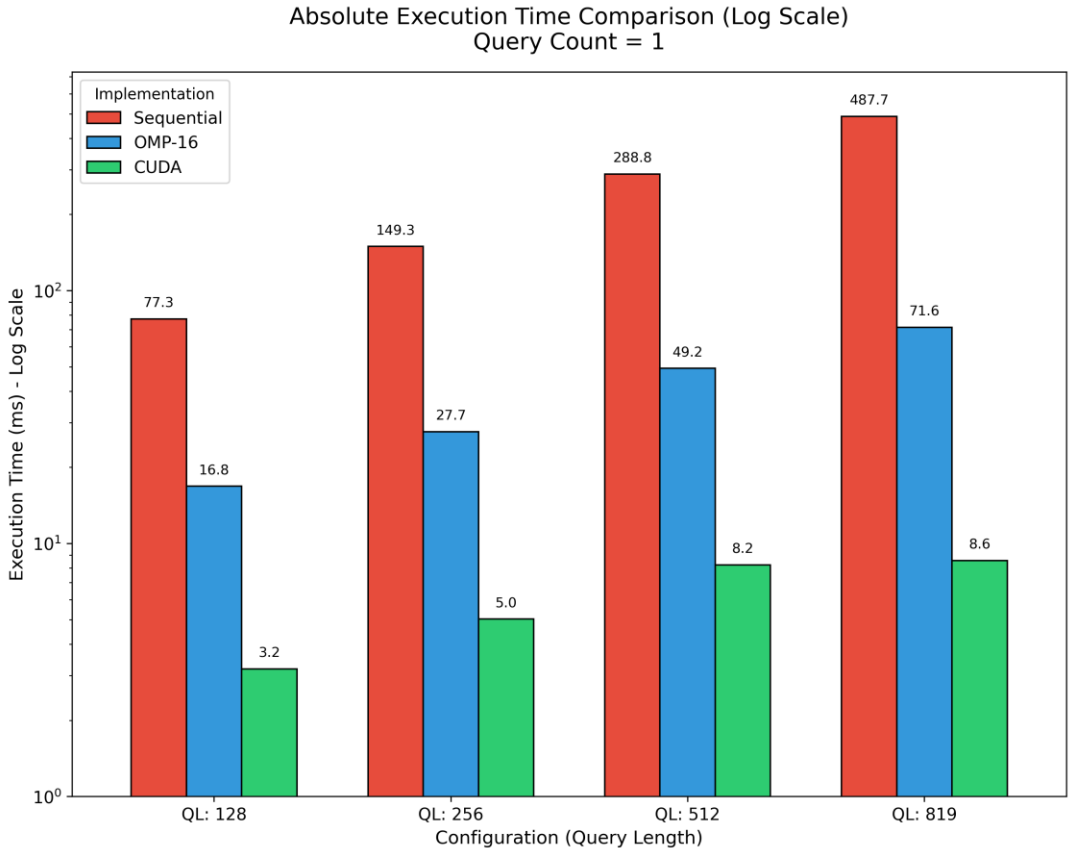


Results: Overall Comparison

- **Sequential:** sequential execution scales linearly and poorly.
- **OMP:** utilizing all the CPU cores and threads provides substantial speedups.
- **CUDA:** CUDA alters the performance tier, dropping execution times into the single-digit milliseconds.

As query length increases, the performance gap between OMP and CUDA widens significantly.

OpenMP is bottlenecked by physical CPU cores, while CUDA becomes more efficient at feeding its execution units from shared memory.



Conclusions

Goal Achieved
Successfully accelerated a 5-channel time series pattern recognition using the SAD metric and parallel approaches.

CPU Results
OMP provides a straightforward path to multi-core speedups. However, careful dynamic scheduling to prevent severe thread load imbalances had to be used.

GPU Results
CUDA delivers the absolute maximum throughput and lowest execution times by abandoning early termination and explicitly managing the GPU's memory hierarchy.